



3

12

1800 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية



شجرة البحث الثنائية BST

عملي مشترك

محتوى مجاني غير مخصص للبيع التجاري

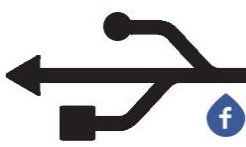
RB Informatics; 16/5/2026

خوارزميات ٢

بسم الله الرحمن الرحيم

مقدمة : بعد أن تعرفنا في المحاضرات السابقة على مفهوم الـ Graph بشكل عام، ثم ركزنا على حالة خاصة الـ Tree وصولاً إلى الشجرة الثنائية (Binary Tree).
نأتي اليوم إلى واحدة من أهم التطبيقات العملية للأشجار الثنائية وأكثرها شيوعاً في علم الحاسوب، ألا وهي شجرة البحث الثنائية (BST).

أهم العناوين الواردة في المحاضرة	
رقم الصفحة	عنوان الفقرة
2	مفهوم الـ BST
2	مفهوم الـ الشجرة المتوازنة
3	الفرق بين البحث في الـ BST و البحث في BT ذات القيم العشوائية
4	العمليات الأساسية على الـ BST وتكويدها



تعريف: نقول عن الـ (Binary Tree) أنها BST إذا تحقق الشرطين التاليين لكل عقدة في الشجرة:

(1) جميع العقد في الشجرة الفرعية اليسرى تحوي قيماً أصغر من قيمة العقدة الحالية.

(2) جميع العقد في الشجرة الفرعية اليمنى تحوي قيماً أكبر من قيمة العقدة الحالية.

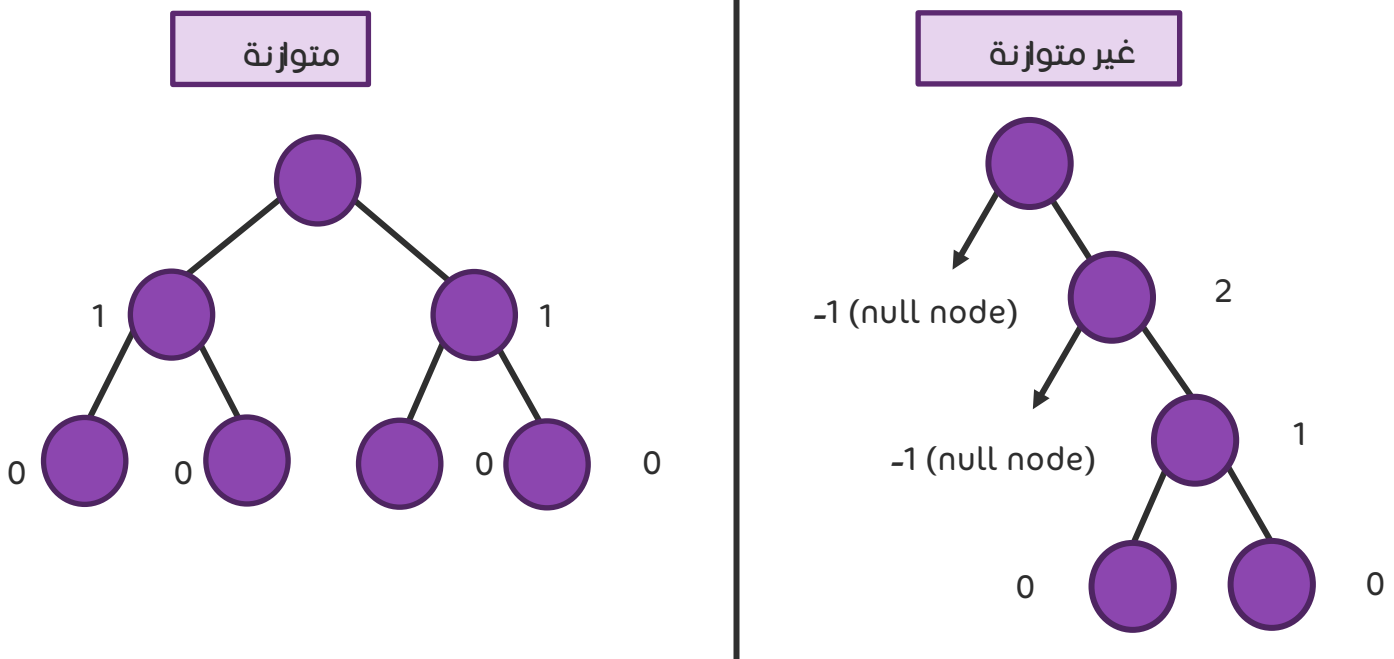
لا يوجد فيها عقد ذات قيم مكررة.

وبالتالي الـ BST هي شجرة ثنائية تخضع لشرط ترتيب معين.

سنتعرف الآن على مفهوم الـ Balanced Tree (الشجرة المتوازنة):

هي الشجرة التي يكون فيها فرق الارتفاع بين الشجرة الفرعية التابعة لابنها الأيسر والشجرة الفرعية التابعة لابنها الأيمن لكل عقدة في الشجرة مساوياً لـ 1 أو 0 أو -1. وأي شجرة لا تحقق ذلك الشرط تكون Unbalanced Tree.

مثال: سنضع ارتفاع الشجرة على يمين وعلى يسار كل عقدة حتى يتضح المعنى.



ملاحظات:

1. نعرف ارتفاع العقدة null بكونه -1 حتى يتفق مفهوم الارتفاع الذي نفهمه (وكذلك مع التعريف العودي) بحيث أن معنى الارتفاع في الشجرة هو بعد الجذر عن الورقة.

2. لو فرضنا عقدة لا أبناء لها يكون بعدها عن الورقة هو (0) كونها هي بذاتها ستكون جذراً وورقة في آن واحد لنفترض أن هذه العقدة اختفت فالبعد سينقص بمقدار عقدة إضافية (العقدة التي اختفت) بالتالي سيصبح -1.

3. الشجر المتوازن وغير المتوازن لا يقتصر على الأشجار الثنائية يمكن تعميمه على أشجار ذوات درجات أعلى

ما سبب تسمية ال Binary Search Tree بهذا الاسم:

لأن الشجرة بطبيعتها تدعم عملية البحث بشكل منهجي لحد ما

ما الفرق بين البحث في ال BST و البحث في الشجرة الثنائية ذات القيم العشوائية:

- في الشجرة الثنائية ذات القيم العشوائية لا يوجد ترتيب للقيم لذلك عند البحث عن قيمة معينة سنضطر للمرور على كافة العقد بأسوأ الأحوال بحيث سيستمر استدعاء التابع على كافة العقد حتى نصل للنتيجة.
- بينما في ال BST فبسبب ترتيب القيم (اليسرى أصغر واليمنى أكبر) سيصبح الأمر مبني على : هل القيمة التي أبحث عنها أصغر من قيمة العقدة الحالية ؟ أم أكبر من قيمة العقدة الحالية ؟ و نميز حالتين:
- (1) في حال كانت القيمة المستهدفة أصغر نذهب إلى الابن الأيسر ونستثني الابن الأيمن.
 - (2) في حال كانت القيمة أكبر نذهب إلى الابن الأيمن ونستثني الابن الأيسر . ونطبق ذلك على كل عقدة حتى نصل إلى الناتج.

نلاحظ أن : البحث بهذه الطريقة يشبه ال binary search بحيث في كل خطوة نستبعد نصف العقد

تقريباً وهذا ما يجعل من التعقيد $O(h)$ حيث h هو ارتفاع الشجرة (بحيث سنضطر للمرور على كل مستوى مرة واحدة فقط) على عكس التعقيد في عملية البحث في الشجرة ذات القيم العشوائية والذي سيكون $O(n)$ حيث n هو عدد العقد.

ولكن سبب شرحنا لمفهوم الشجرة المتوازنة يرجع لنقطة وهي في حال كانت ال BST متوازنة هنا سيكون: $h = \log_2(n)$ تقريباً

حيث في كل عملية سنستثني نصف عدد العقد تقريباً مما يجعل التعقيد $O(\log_2(n))$

بينما إذا كانت ال BST غير متوازنة فمن الممكن ل h في أسوأ حالة أن تساوي ال n (شجرة خطية) ، هنا ستفقد ميزة البحث الثنائي لأن كل عقدة لها ابن واحد فقط وبالتالي سيصبح البحث خطي بتعقيد $O(n)$

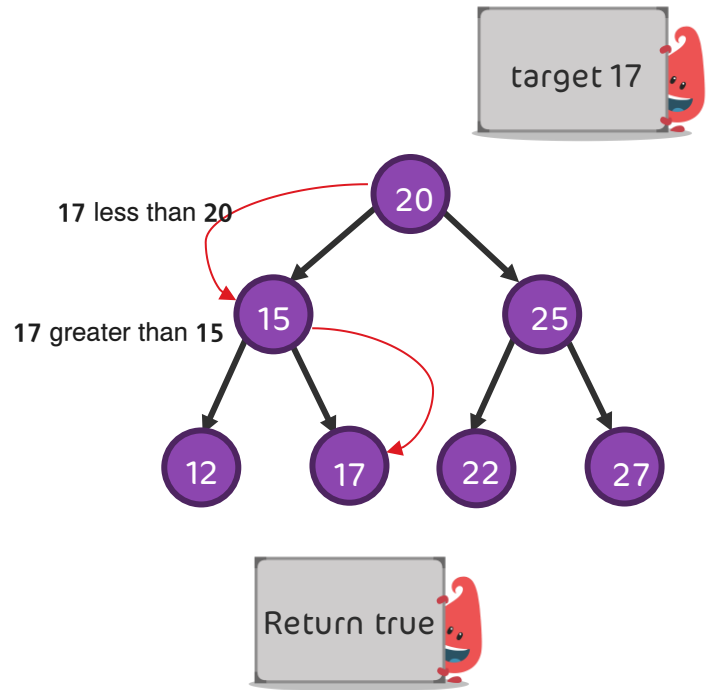
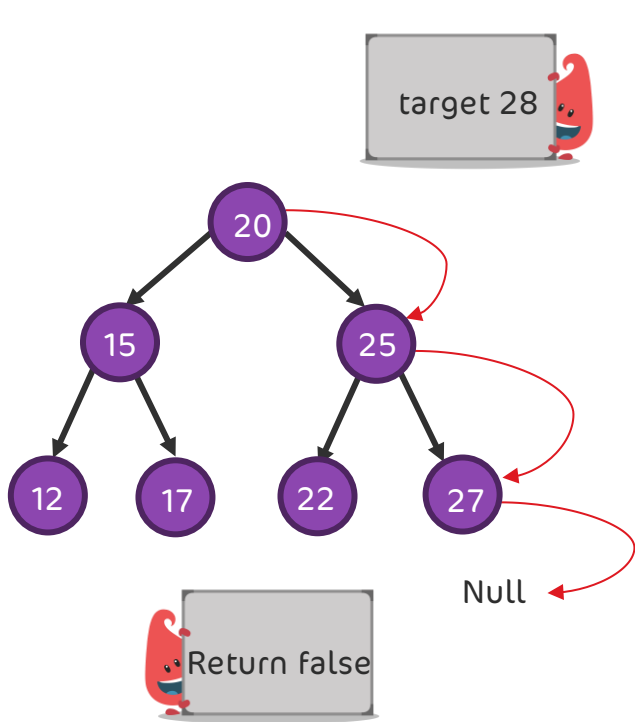
لا يمكن لـ BST العادية أن تضمن التوازن وهذه أهم المشكلات التي سنتحدث عنها في موضوعات المحاضرات اللاحقة وكيفية حلها.

الآن سنتحدث بشكل عام عن العمليات الأساسية التي يمكن إجراؤها ضمن هذه الشجرة :

يدعم الـ BST ثلاث عمليات أساسية هي البحث (search) والادراج (insert) والحذف (delete) بحيث هذه العمليات تعتمد على قاعدة الترتيب (اليسار أصغر، اليمين أكبر)

1. أولاً: عملية البحث (Search) :

الفكرة: نبدأ من الجذر ونقارن القيمة المطلوبة مع قيمة العقدة الحالية فإذا كانت القيمة المطلوبة أصغر من قيمة العقدة الحالية نتجه يساراً وإذا كانت أكبر نتجه يميناً وإذا تساوت معها نعيد True وإذا وصلنا إلى null نعيد False



والآن سنقوم بكتابة تابع الـ Search بطريقتين عودية و تكرارية:

حيث بالكود التالي سنكتبه بشكل عودي recursive :

```

2 class BinaryTree {
3     Node root;
4     class Node {
5         int value;
6         Node left, right;
7         Node(int value) {
8             this.value = value;
9             this.left = this.right = null;
10        }
11    }
12
13    public boolean search (Node root , int key){
14        if(root == null)
15            return false;
16        if(root.value == key)
17            return true;
18        else if (key < root.value)
19            return search(root.left, key);
20        else
21            return search(root.right, key);
22    }
23 }

```

```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4
5         bt.root = bt.new Node(20);
6
7         bt.root.right = bt.new Node(25);
8         bt.root.right.left = bt.new Node(22);
9         bt.root.right.right = bt.new Node(27);
10
11        bt.root.left = bt.new Node(15);
12        bt.root.left.right = bt.new Node(17);
13
14        bt.root.left.left = bt.new Node(12);
15
16        System.out.println(bt.search(bt.root,
17                                     17)); // true
18        System.out.println(bt.search(bt.root,
19                                     28)); // false
18    }
19 }

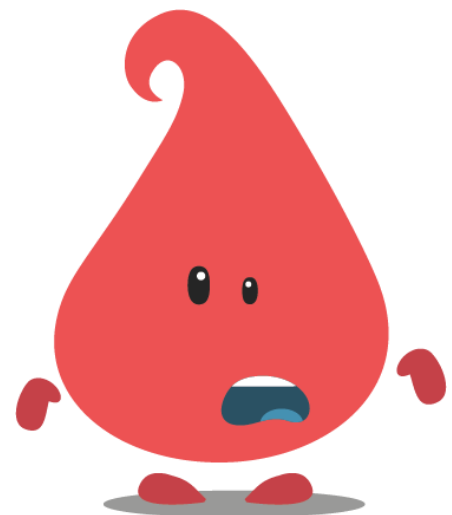
```

تابع ال search بالشكل التكراري iterative :

```

2 public boolean search(Node root, int
key) {
3     while (root != null) {
4         if (key > root.value)
5             root = root.right;
6         else if (key < root.value)
7             root = root.left;
8         else
9             return true;
10    }
11    return false;
12 }

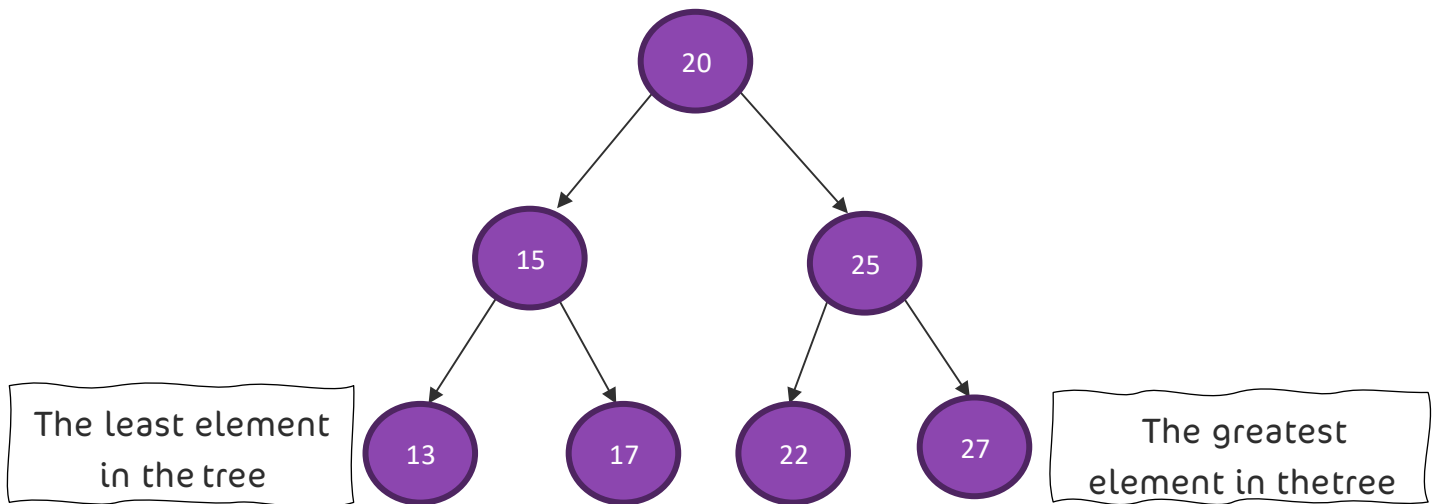
```



■ إيجاد أصغر عنصر (Find Min):

الفكرة : أصغر عنصر في الـ BST هو أقصى عقدة في الجهة اليسرى (نستمر يساراً حتى نصل إلى (null) .

■ إيجاد أكبر عنصر (Find Max): الفكرة : أكبر عنصر في الـ BST هو أقصى عقدة في الجهة اليمنى (نستمر يميناً حتى نصل إلى (null) .



”وتهدينا الحياة أضواء في آخر النفق

تدعونا كي نخسى ألما عشناه



```

1 class BinaryTree {
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             this.left = this.right = null;
9         }
10    }
11
12    public int minElement(Node root){
13        if(root.left==null)
14            return root.value;
15        return minElement(root.left);
16    }
17
18    public int maxElement(Node root){
19        if(root.right==null)
20            return root.value;
21        return maxElement(root.right);
22    }
23 }

```

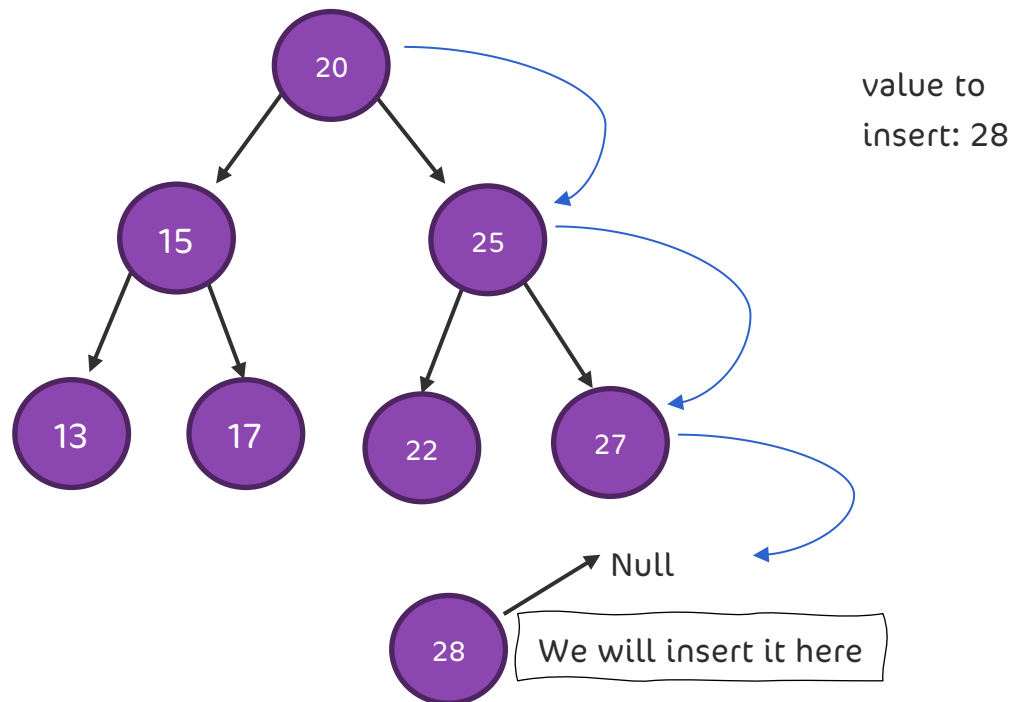
```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(20);
5         bt.root.right = bt.new Node(25);
6         bt.root.right.left = bt.new Node(22);
7         bt.root.right.right = bt.new Node(27);
8         bt.root.left = bt.new Node(15);
9         bt.root.left.right = bt.new Node(17);
10        bt.root.left.left = bt.new Node(13);
11
12        System.out.println(bt.minElement(bt.root));
13        System.out.println(bt.maxElement(bt.root));
14    }
15 }

```

2. ثانياً: عملية الإدراج (Insert) :

- الفكرة: نبدأ من الجذر ونطبق نفس منطق البحث للوصول إلى المكان المناسب للعقدة الجديدة، حيث في كل خطوة نقارن القيمة المراد إدراجها مع قيمة العقدة الحالية: إذا كانت أصغر نتجه يساراً وإذا كانت أكبر نتجه يميناً، ونستمر حتى نصل إلى عقدة فارغة (null) وهناك ننشئ العقدة الجديدة ونضيفها كورقة leaf بحيث لا نسمح بتكرار القيم، فإذا وجدنا القيمة موجودة بالفعل لا نقوم بأي إجراء.



```

1 class BinaryTree {
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             this.left = this.right = null;
9         }
10    }
11
12    public Node insert (Node root, int val){
13        if (root == null)
14            return new Node(val);
15        if (root.value > val)
16            root.left = insert(root.left, val);
17        else
18            root.right = insert(root.right, val);
19        return root;
20    }
21 }

```

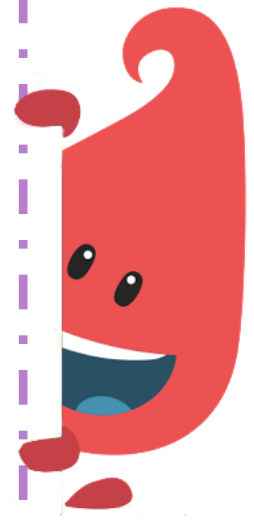
في الكود التالي سنكتب تابع ال insert بالشكل العودي recursive :

```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(20);
5         bt.root.right = bt.new Node(25);
6         bt.root.right.left = bt.new Node(22);
7         bt.root.right.right = bt.new Node(27);
8         bt.root.left = bt.new Node(15);
9         bt.root.left.right = bt.new Node(17);
10        bt.root.left.left = bt.new Node(13);
11
12        bt.insert(bt.root, 28);
13
14        System.out.println(bt.root.right.right.right.value); //28
15    }
16 }

```

```
1 public Node insert(Node root, int key) {  
2     Node node = new Node(key);  
3     if (root == null) {  
4         return node;  
5     }  
6     Node prev = null;  
7     Node temp = root;  
8     while (temp != null) {  
9         prev = temp;  
10        if (temp.value > key) {  
11            temp = temp.left;  
12        } else if (temp.value < key) {  
13            temp = temp.right;  
14        } else {  
15            return root;  
16        }  
17    }  
18    if (prev.value > key)  
19        prev.left = node;  
20    else  
21        prev.right = node;  
22    return root;  
23 }
```



"تصرف كما لو أنه من المستحيل أن تفشل"

3. ثالثاً: عملية الحذف: (Delete) :

■ الفكرة: في حال أردنا حذف عقدة من الـ BST للتخلص من هذا العنصر يجب أن تبقى الشجرة محافظة على الشروط الخاصة بالـ BST كما في عملية الإضافة. وبالتالي عند العثور على العقدة المراد حذفها هناك ثلاث حالات يجب معالجتها بناءً على عدد أبناء هذه العقدة.

1. الحالة الأولى: العقدة المراد حذفها ورقة (leaf) بمعنى ليس لها أبناء من الجهتين .
هنا ببساطة نحذفها مباشرة، أي نجعل الأب يشير إلى null مكانها

الحالة الثانية: العقدة المراد حذفها لها ابن واحد... عندئذ نجعل ابنها يحل مكانها فحسب.

2. الحالة الثالثة: العقدة المراد حذفها لها اثنان.

المشكلة: لو حذفناها مباشرة سنفقد الاتصال بفرع كامل و لا يمكننا رفع أحد الابنين مباشرة لأن ذلك يخالف ترتيب الـ BST .

الحل: نبحث عن العقدة الأقرب للعقدة المراد حذفها ونأخذ قيمتها و نضعها في العقدة المطلوب حذفها ثم نحذف تلك العقدة (العقدة الأقرب للعقدة المراد حذفها) من مكانها الأصلي و بهذه الطريقة سيبقى الترتيب صحيحاً .

ما هي أقرب عقدة للعقدة الحالية في الترتيب؟

هناك عقدتان:

(1) العنصر التالي: نجده في الشجرة اليمنى وتحديداً أصغر قيمة فيها . لماذا؟ لأن أصغر قيمة في الشجرة اليمنى هي أكبر من العقدة الحالية ولكنها أصغر من كل قيم الشجرة اليمنى . كيف نصل إليه؟ نتقل إلى الابن الأيمن ثم ننزل إلى أقصى اليسار

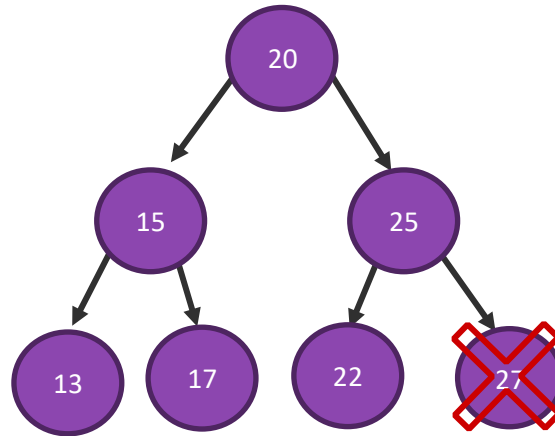
(2) العنصر السابق: نجده في الشجرة اليسرى وتحديداً أكبر قيمة فيها، لماذا؟ لأن أكبر قيمة في الشجرة اليسرى هي أصغر من العقدة الحالية ولكنها أكبر من كل قيم الشجرة اليسرى . كيف نصل إليه؟ نتقل إلى الابن الأيسر ثم ننزل أقصى اليمين.

1. الحالة الأولى: فقط نسند العقدة لـ null .

2. الحالة الثانية: فقط نسند العقدة للابن الموجود.

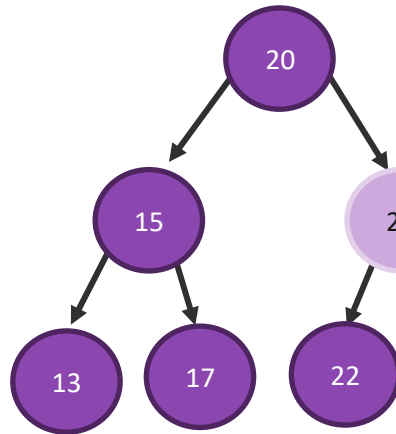
3. الحالة الثالثة: نبحث عن أحد العنصرين المميزين الذين ذكرناهما ثم نأخذ قيمته ونضعها في العقدة التي طلب حذفها ومن ثم نذهب إلى مكان هذا العنصر المميز ونحذفه.

■ بالكود القادم سوف نذهب للعنصر التالي للعقدة المراد حذفها ونأخذ قيمته ونضعها في هذه العقدة ثم نحذف هذا العنصر



value to
delete: 27

Assign this to null



This would be 22

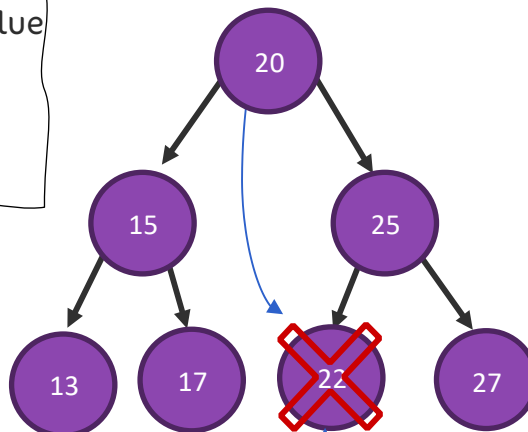
value to
delete: 25

Assign the child to the parent

This would be null

Value to
Delete : 20

Assign this to minimum value
of the right tree
Then delete the minimum
right node



This would be null



```

1 class BinaryTree {
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             this.left = this.right = null;
9         }
10    }
11 }

```

```

1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node(20);
5         bt.root.right = bt.new Node(25);
6         bt.root.right.left = bt.new Node(22);
7         bt.root.right.right = bt.new Node(27);
8         bt.root.left = bt.new Node(15);
9         bt.root.left.right = bt.new Node(17);
10        bt.root.left.left = bt.new Node(13);
11    }
12    System.out.println(bt.root.value); //20
13    bt.delete(bt.root, 20);
14    System.out.println(bt.root.value); //22
15 }
16 }

```

```

1 public Node minElement(Node root){
2     if(root.left!=null)
3         return minElement(root.left);
4     else
5         return root;
6 }
7
8 public Node delete (Node root,int val){
9     // if we don't find the val
10    if (root == null)
11        return null;
12    if(val < root.value)
13        root.left = delete(root.left,val);
14    else if(val > root.value)
15        root.right = delete(root.right,val);
16    else {
17        // if no children
18        if(root.left==null && root.right==null)
19            return null;
20        //if only one child
21        else if (root.left==null)
22            return root.right;
23        else if (root.right ==null)
24            return root.left;
25        //if two children
26        else {
27            Node node = minElement(root.right);
28            root.value = node.value;
29            root.right=delete(root.right,node.value);
30        }
31    }
32    return root;
33 }}

```